

Pageflow

Full usage guide — copy-paste recipes for every feature, client & server.

Pageflow is the SPA bridge (an Inertia.js v2 fork) for the AlfacodeTeam PhpServicePlatform. You write normal server controllers that return a **component name + props**; the React client swaps components in place. This guide is the practical companion to the architecture guide — every section is something you can paste and run. Server snippets are `PHP`, client snippets are `React/TS`.

Client packages: `@pageflow/core` , `@pageflow/react` · Server: `plugins/Pageflow/` · Architecture & security: `PAGEFLOW_GUIDE.pdf`

1. Setup

1.1 Environment (`.env`)

```
PAGEFLOW_VERSION="1"                # bump to force clients onto new assets
PAGEFLOW_ROOT_VIEW="/abs/plugins/Pageflow/resources/views/app.php"
PAGEFLOW_APP_ID="app"                # root element id
PAGEFLOW_CSRF_COOKIE="hkm_session"   # must be in Cookie encrypt_exempt
PAGEFLOW_STREAM_MAX_SECONDS="300"    # SSE connection lifetime cap
```

1.2 Register the plugin (project `bootstrap/app.php`)

```
return $builder
->withModules([ Plugins\Pageflow\Provider::class, /* ... */ ])
->build();
```

1.3 Client entry (`main.tsx`)

```
import { createPageflowApp, resolvePageComponent, installCsrfAutoRefresh } from '@pageflow/react'
import { createRoot } from 'react-dom/client'

installCsrfAutoRefresh() // long-lived tabs auto-recover expired CSRF tokens

createPageflowApp({
  resolve: (name) => resolvePageComponent(name, import.meta.glob('./Pages/**/*.tsx')),
  setup: ({ el, App, props }) => createRoot(el).render(<App {...props} />),
})
```

1.4 Root layout (`resources/views/app.php`)

Rendered in an isolated scope with `$FLOW_PAGE` , `$FLOW_CSRF` , `$FLOW_APP_ID` . The one line that matters emits the boot element:

```
<!DOCTYPE html>
<html><head>
  <meta charset="utf-8">
  <?php if ($FLOW_CSRF !== ''): ?>
    <meta name="csrf-token" content="<?= htmlspecialchars($FLOW_CSRF, ENT_QUOTES) ?>">
  <?php endif; ?>
  <script type="module" src="/build/app.js" defer></script>
```

```
</head><body>
  <?=$FLOW_PAGE->mount($FLOW_APP_ID) ?> <!-- <div id="app" data-page="{...}"> -->
</body></html>
```

2. Pages & navigation

2.1 A controller returns a page

```
final class UserController {
    public function __construct(private readonly PageflowResponder $pageflow, private readonly UserSer

    public function index(Request $request): Response {
        return $this->pageflow->render($request, 'Users/Index', ['users' => $this->users->all()]);
    }
    public function show(Request $request, string $id): Response {
        return $this->pageflow->render($request, 'Users/Show', ['user' => $this->users->find($id)]);
    }
}
```

Routes live in the plugin's `module.json` or the project's `proj.json` — never in PHP.

2.2 The page component

```
import { usePage, Head, Link } from '@pageflow/react'

export default function Index() {
    const { users } = usePage().props
    return (<
        <Head title="Users" />
        {users.map((u) => <Link key={u.id} href={`/${u.id}`}>{u.name}</Link>)}
    </>)
}
```

2.3 Navigating from code

```
import { router } from '@pageflow/react'
router.get('/users', { page: 2 }) // query params
router.post('/users', { name: 'Sam' }) // preserveState defaults true for POST/PUT/PATCH/DELETE
router.visit('/users', { replace: true, preserveScroll: true })
router.reload({ only: ['users'] }) // partial reload
```

`<Link>` options: `method`, `data`, `only` / `except`, `preserveScroll`, `replace`, `prefetch`
 (`"hover"` | `"mount"` | `"click"`), `cacheFor`, `as` .

3. Reading data & shared props

```
// bootstrap — resolved per request, present on EVERY page
pageflow_share('auth', fn($request) => $request->identity()?->userId);
pageflow_share('appName', fn() => 'HKM');
```

```
const { auth, appName, users } = usePage().props
```

```
// typed: const { users } = usePage<{ users: User[] }>().props
```

4. Forms

4.1 `useForm` hook

```
import { useForm } from '@pageflow/react'

const form = useForm({ email: '', password: '' })

<form onSubmit={e => { e.preventDefault(); form.post('/login') }}>
  <input value={form.data.email} onChange={e => form.setData('email', e.target.value)} />
  {form.errors.email && <span>{form.errors.email}</span>}
  <button disabled={form.processing}>Log in</button>
</form>
```

State: `data`, `errors`, `hasErrors`, `processing`, `progress`, `isDirty`, `wasSuccessful`, `recentlySuccessful`. **Methods:** `setData`, `get/post/put/patch/delete/submit`, `reset`, `clearErrors`, `setError`, `transform`, `cancel`.

```
// submit options
form.post('/users', {
  resetOnSuccess: true,           // or ['password'] – reset fields after success
  resetOnError: ['password'],
  onSuccess: (page) => {}, onError: (errors) => {},
})
form.transform((d) => ({ ...d, email: d.email.trim() })) // mutate before send
```

4.2 `<Form>` component (declarative)

```
import { Form } from '@pageflow/react'

<Form action="/users" method="post" resetOnSuccess validateOn="blur">
  ({ errors, processing }) => (<
    <input name="name" /> {errors.name && <span>{errors.name}</span>}
    <input name="avatar" type="file" /> // files handled automatically
    <button disabled={processing}>Save</button>
  </>)}
</Form>
```

5. Validation & precognition

5.1 Server-driven errors (automatic)

Throw a `ValidationException` from your DTO; the errors flow back to the form — no manual wiring:

```
public function store(Request $request): Response {
    $dto = CreateUserDTO::fromRequest($request); // throws ValidationException (422 + fields)
    if (pageflow_precognition($request)) {
        return $this->pageflow->precognitionSuccess(); // validate-only: stop before writes
    }
    $this->users->create($dto);
    return Response::redirect('/users');
}
```

Precognition safety: a precognitive request runs your validation but MUST NOT write. Always short-circuit with `pageflow_precognition($request)` before side effects. The `precognition` request attribute lets repos/services double-check.

5.2 Live validation (client)

```
const data = { email, password }
const pre = usePrecognition('post', '/register', () => data) // debounced 300ms

pre.errors.email && <span>{pre.errors.email}</span>
```

Or fully declarative with `<Form validateOn="blur">` (§4.2). Add the `throttle` route filter on precognition routes.

6. Partial reloads & lazy data

Tool	Use	Example
<code>only/except</code>	reload some props	<code>router.reload({ only: ['stats'] })</code>
<code>Deferred</code>	render now, load slow data after	<code><Deferred data="report" fallback={<Spin/>}>{...}</Deferred></code>
<code>WhenVisible</code>	load on scroll-into-view	<code><WhenVisible data="comments">{...}</WhenVisible></code>
<code>usePoll</code>	auto re-fetch every N ms	<code>usePoll(5000, { only: ['feed'] })</code>
<code>usePrefetch</code> / <code><Link prefetch></code>	fetch before click	<code><Link href="/x" prefetch="hover" /></code>
<code>useRemember</code>	keep local state across nav	<code>const [v,set]=useRemember('', 'draft')</code>

Deferred groups are **batched** into a single reload request ($N \rightarrow 1$) automatically.

7. Reactive props (secure realtime, no polling)

The server pushes only "these keys are stale" — never data. The client re-fetches through the authenticated pipeline, so realtime leaks nothing.

7.1 Emit (server, after a commit)

```
final class OrderService {
  public function __construct(private readonly PageflowChannel $channel, /* ... */) {}

  public function place(...): ... {
    // ... create + commit ...
    $this->channel->touch("t:{$tenantId}:dashboard", ['orders', 'stats']);
  }
}
```

Or subscribe an EventBus integration event to touch the channel.

7.2 Subscribe (client)

```
import { useReactiveProps } from '@pageflow/react'

useReactiveProps(['orders', 'stats'], { channel: 'dashboard' })
// when the server touches those keys → an authorized partial reload runs automatically
```

Endpoint `GET /pageflow/stream` ships ready-made (auth + tenant-scoped). Run under OpenSwoole.

8. Auth & permissions (UX gating)

```
import { useAuth, Can, useFlushOnIdentityChange } from '@pageflow/react'

const { userId, can, hasRole } = useAuth()
<Can permission="invoice:create"><NewInvoiceButton /></Can>
<Can anyOf={['post:edit', 'post:moderate']} fallback=<ReadOnly/>>{...}</Can>

// once near the app root – purges caches on login/logout/tenant-switch
useFlushOnIdentityChange()
```

```
// optional – send capability booleans instead of raw permission names
pageflow_auth_projection(fn(?Identity $id) => [
  'userId' => $id->userId ?? '',
  'canManage' => (bool) $id->hasPermission('admin:manage'),
]);
```

Server is the authority. `useAuth` / `<Can>` only hide UI — enforce every mutation in the Service layer.

9. Head, CSRF, offline, types

9.1 Head

```
<Head title="Dashboard"><meta name="description" content="..." /></Head>
```

9.2 CSRF (mostly automatic)

The `<meta name="csrf-token">` in your layout is read by the client and sent as `X-CSRF-Token` on mutations (same-origin only). `installCsrfAutoRefresh()` (§1.3) recovers expired tokens via `GET /pageflow/csrf`. No per-request work needed.

9.3 Offline (opt-in)

```
registerPageflowSW() // client, once – serve pageflow-sw.js from web root
```

```
// opt a PUBLIC page (no user data) into the offline cache
return $this->pageflow->render($request, 'Marketing/Home', $props, cacheable: true);
```

Authenticated pages are never cached by default. Call `clearPageflowSWCache()` on logout (or use `useFlushOnIdentityChange`).

9.4 End-to-end types

```
hkm pageflow:types --pages=resources/js/Pages --out=resources/js/pageflow.d.ts
```

Generates shared-prop typings + a page registry, so `usePage<T>()` is typed and new pages autocomplete. Re-run after adding pages.

10. API cheatsheet

Import (<code>@pageflow/react</code>)	What
<code>router</code>	<code>get/post/put/patch/delete/visit/reload/poll/prefetch/flushAll</code>
<code><Link></code> , <code><Head></code> , <code><Form></code> , <code><Can></code> , <code><Deferred></code> , <code><WhenVisible></code>	components
<code>useForm</code> , <code>usePage</code> , <code>usePrecognition</code> , <code>useReactiveProps</code> , <code>useAuth</code>	core hooks
<code>useDirtyGuard</code> , <code>usePoll</code> , <code>usePrefetch</code> , <code>useRemember</code> , <code>useFlushOnIdentityChange</code>	UX hooks
<code>createPageflowApp</code> , <code>installCsrfAutoRefresh</code> , <code>registerPageflowSW</code> , <code>clearPageflowSWCache</code>	bootstrap

Server	What
<code>PageflowResponder::render(\$req,\$component,\$props,\$loadPage,\$cacheable)</code>	the response
<code>PageflowResponder::precognitionSuccess()</code>	validate-only OK
<code>PageflowChannel::touch(\$channel,\$keys)</code>	emit reactive signal

pageflow_share() , pageflow_auth_projection() , pageflow_precognition()	helpers
---	-------------------------

Pageflow Usage Guide · AlfacodeTeam PhpServicePlatform · see [PAGEFLOW_GUIDE.pdf](#) for architecture, wire protocol & the security hardening ledger. Pageflow is a fork of Inertia.js (MIT).